30 DE ENERO DE 2026

INFORME PRÁCTICA 2

ENTRADA Y SALIDA EN C

GUILLERMO FUENTES, IRENE MORALES

Índice

Introducción ____ 2
Código y Desarrollo Práctico ____ 2
Manejo de puertos ____ 2
Diseño 1: LEDs individuales ____ 3
Manejo del pulsador ____ 4
Detección de flancos ____ 5
Diseño 2: Secuencia ____ 6
Conclusión ____ 7

Introducción

En esta práctica se ha trabajado con la tarjeta montada en la práctica anterior para ver cómo funcionan los puertos de entrada y salida digitales del microcontrolador mediante programación en lenguaje C. Se ha interactuado con los LEDs conectados al puerto C y con el pulsador situado en el pin RB5 del puerto B.

Para ello, se han configurado los registros TRISx, LATx y PORTx para controlar el hardware de forma precisa. También, se ha implementado la detección de flancos para evitar lecturas erróneas en pulsadores mecánicos.

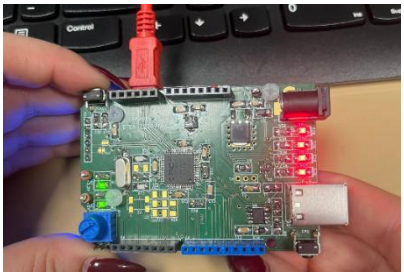
Código y Desarrollo Práctico

Manejo de puertos

Para empezar con la práctica configuramos el puerto C para controlar LEDs. Al escribir TRISC = 0xFFF0, dejamos los bits RC0–RC3 como salidas y el resto de pines RC4–RC15 como entradas. A continuación, con LATC = 0xFFF0 fijamos el estado inicial de esas salidas poniendo RC0–RC3 a 0 por lo que los LEDs conectados a RC0–RC3 quedan encendidos. Finalmente, el while(1); mantiene el microprocesador en un bucle infinito sin ejecutar más instrucciones, de modo que los LEDs se quedan permanentemente en ese estado inicial.

```c
#include <xc.h>

int main(void) {

    TRISC = 0xFFF0;
    LATC = 0xFFF0;
    while (1)
        ;
    return 0;
}
```

Código .C para el Manejo de Puertos



Diseño 1: LEDs individuales

En este segundo programa continuamos con la misma configuración del puerto C. En nuestro caso tenemos dos apartados que nos piden que encendamos una combinación de LEDs diferente.

1. En el primer apartado nos pide que encendamos solamente el LED conectado a RC0 por lo que con LATC = 0xFFFE ponemos a 0 el bit que maneja dicho pin.
2. Para el segundo apartado nos piden que encendamos RC3 y RC1, por lo que con la línea LATC = 0xFFF5 tendremos en el último cuarteto "0101" por lo que se pondrán a 0 el bit 1 y 3 y se encenderán los LEDs mencionados.

El TRISx no debe de ser modificado ya que lo único que cambia es el estado de los LEDs, no la naturaleza de los pines conectados, en otras palabras, se mantienen como salidas.

```c
#include <xc.h>

int main(void) {

    TRISC = 0xFFF0;
   // LATC = 0xFFFE;

    LATC = 0xFFF5;

    while (1)
        ;
    return 0;

}
```

Código .C para Diseño 1

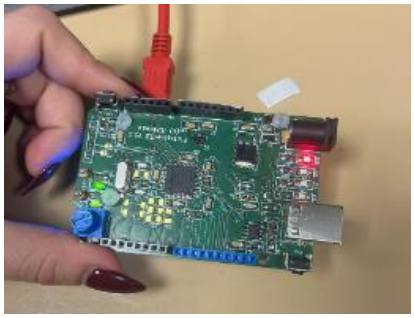


Figura 2: 1110 = 14

Figura 3: 0101 = 5

Manejo del pulsador

En este tercer apartado el objetivo es poder usar el pulsador que se encuentra en el puerto B, concretamente en el pin 5. Con la línea #define PIN_PULSADOR 5 lo que hacemos es definir que el pulsador se encuentra en el bit 5. Con TRISC = 0xFFF0 mantenemos los LEDs como salidas. Seguimos configurando con LATC = 0x000F lo que nos permite que los bits que corresponden a los LEDs estén a 1, conllevará que los LEDs estén apagados inicialmente. Para configurar como entrada los pulsadores, en el registro B ponemos todos los bits como entrada, es decir, a 1. Dentro del bucle while(1), lo que hacemos es leer solamente el bit del pulsador, si este es 0, es decir, está pulsado, la salida enciende el LED 0, en caso contrario se mantendrá apagado.

```c
#include <xc.h>
#define PIN_PULSADOR 5

int main(void) {

    int pulsador;
    TRISC = 0xFFF0;
    LATC = 0x000F;
    TRISB = 0xFFFF;

    while (1){
        pulsador = (PORTB >> PIN_PULSADOR)&1;
        if (pulsador==0) {
            LATC &= ~1;
        }
        else {
          LATC |= 1;
        }
    }
    return 0;
}
```

Código .C para el manejo del pulsador

Figura 4: 1111 = 15

Figura 5: 1110 = 14

Detección de flancos

En este programa se implementa la detección de flancos para contar el número de pulsaciones realizadas sobre el pulsador conectado al pin RB5. Para ello, se comparan el estado actual y el estado anterior del pulsador, lo que permite detectar únicamente nuevas pulsaciones y evitar múltiples conteos mientras el botón permanece presionado.

El contador se incrementa cuando se detecta un flanco de bajada, es decir, cuando el estado del pulsador cambia de '1' a '0'. El valor del contador se muestra en binario mediante los cuatro LEDs conectados al puerto C. Dado que los LEDs son activos a nivel bajo, se aplica una inversión lógica antes de escribir el valor en el registro LATC.

Como solo se dispone de cuatro LEDs, el contador se reinicia al superar el valor 15.

```c
#include <xc.h>
#define PIN_PULSADOR 5

int main(void) {

    int pulsador_ant, pulsador_act;
    int cuenta = 0;

    TRISC = 0xFFF0;
    LATC = 0xFFFF;
    TRISB = 0xFFFF;

    pulsador_ant = (PORTB >> PIN_PULSADOR)&1;

    while (1) {

        pulsador_act = (PORTB >> PIN_PULSADOR)&1;
        if ((pulsador_act != pulsador_ant) && (pulsador_act == 0)) {
            cuenta++;
            if (cuenta > 15) {
                cuenta = 0;
            }
            LATC = ~cuenta;
        }
        pulsador_ant = pulsador_act;
    }
}
```

Código .C para la detección de flancos

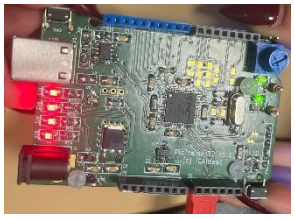


Figura 6: 1111 = 15

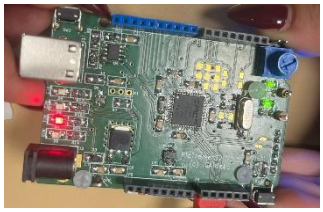


Figura 7: 0010 = 2

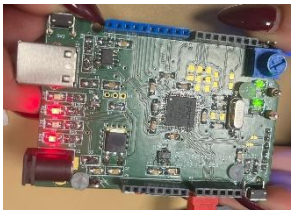


Figura 8: 0101 = 5

Diseño 2: Secuencia

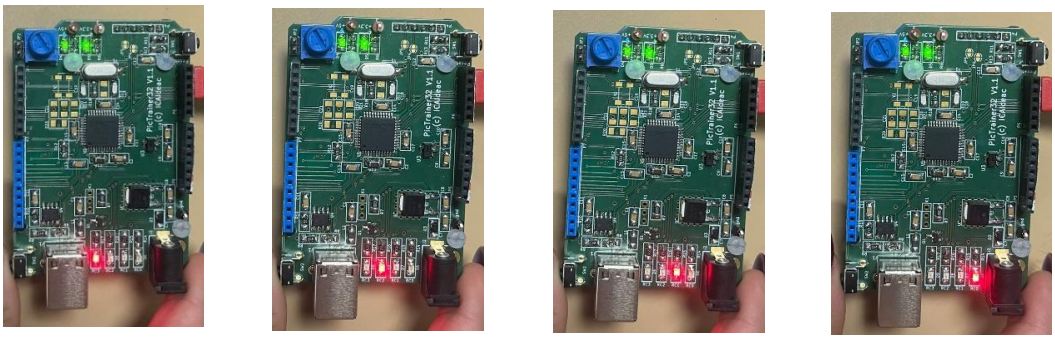
En este apartado se implementa un desplazamiento secuencial del LED encendido utilizando los cuatro LEDs conectados al puerto C. Inicialmente se enciende el LED RC0 y, con cada pulsación del pulsador conectado al pin RB5, el LED activo se desplaza una posición hacia la izquierda. Al alcanzar el LED RC3, la secuencia se reinicia desde el LED inicial.

El programa se basa en la detección de flancos de bajada del pulsador, de forma que cada desplazamiento corresponde a una única pulsación. La posición del LED encendido se controla mediante una variable que se incrementa en cada flanco detectado y se reinicia cuando supera el número de LEDs disponibles.

El encendido del LED correspondiente se realiza mediante el registro LATC, aplicando la lógica inversa necesaria al tratarse de LEDs activos a nivel bajo.

```c
#include <xc.h>
#define PIN_PULSADOR 5

int main(void) {

    int pulsador_ant, pulsador_act;
    int posicion = 0;

    TRISC = 0xFFF0;
    LATC = 0xE;
    TRISB = 0xFFFF;

    pulsador_ant = (PORTB >> PIN_PULSADOR)&1;

    while (1) {

        pulsador_act = (PORTB >> PIN_PULSADOR)&1;
        if ((pulsador_act != pulsador_ant) && (pulsador_act == 0)) {
            posicion++;
            if (posicion > 3) {
                posicion = 0;
            }
            LATC = 0x000F&~(1<<posicion);
        }
        pulsador_ant = pulsador_act;
    }
    return 0;
}
```

Código .C para la secuencia



Figuras 9, 10, 11 y 12: Desplazamiento secuencial

Conclusión

En esta práctica se ha trabajado el manejo de los puertos de entrada y salida digitales del microcontrolador mediante programación en lenguaje C, permitiendo controlar LEDs y leer el estado de un pulsador. Se han configurado correctamente los registros TRISx, LATx y PORTx, utilizando operaciones a nivel de bit para interactuar con el hardware.

Además, se ha implementado la detección de flancos para gestionar de forma correcta las pulsaciones del botón, evitando conteos erróneos. A partir de esta técnica se han desarrollado aplicaciones sencillas como el conteo de pulsaciones y el desplazamiento secuencial de LEDs.

Los resultados observados en la placa, reflejados en las imágenes incluidas en el informe, confirman el correcto funcionamiento de los programas desarrollados y el cumplimiento de los objetivos propuestos en la práctica.